

How To Run The Project

Copy each block into the terminal, one line at a time. Every command has a comment saying what it does.

Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transactions · Dept. of ISE, Global Academy of Technology

Before starting. Python 3.10 or newer must be installed. Nothing else is needed: no database, no server, no special hardware. The whole project has only three library dependencies and installs in about thirty seconds.

On a Mac

USE THE TERMINAL APP

```
# STEP 1 – Go into the code folder.
# Tip: type "cd " then drag the "code" folder onto the Terminal window
# and the full path appears automatically.
cd "/Users/YOURNAME/Desktop/project 2 global/code"

# STEP 2 – Install the three required libraries. One time only, about 30 seconds.
pip3 install -r requirements.txt

# STEP 3 – Create the sample datasets. Takes 2 seconds, produces 125 MB.
python3 -m datasets.generate --dataset all --size medium

# STEP 4 – THE MAIN DEMO. Shows the whole story: upload, audit, add an owner,
# remove an owner, then SELL the dataset. About 1 minute.
python3 -m demo.run_demo

# STEP 5 – THE BEST ONE TO SHOW. Attacks the older research method and breaks
# it, then runs the identical attack on ours, where it fails. About 1 minute.
python3 -m attacks.collusion

# STEP 6 – Second attack: somebody faking co-ownership of your data. Blocked.
python3 -m attacks.rogue_key

# STEP 7 – Third attack: the cloud secretly corrupting data. Always caught.
python3 -m attacks.tampering

# STEP 8 – Run all 41 tests to prove the code is correct. About 1 minute.
python3 -m pytest tests/ -v
```

On Windows

USE THE TERMINAL INSIDE VS CODE

In VS Code, click **Terminal** in the top menu, then **New Terminal**. A panel opens at the bottom of the window. Type the commands there.

```
# STEP 1 – Go into the code folder.
# Tip: right-click the "code" folder, choose "Copy as path", then paste it.
cd "C:\Users\YOURNAME\Desktop\project 2 global\code"

# STEP 2 – Install the three required libraries. One time only, about 30 seconds.
pip install -r requirements.txt

# STEP 3 – Create the sample datasets. Takes 2 seconds, produces 125 MB.
python -m datasets.generate --dataset all --size medium

# STEP 4 – THE MAIN DEMO. Shows the whole story: upload, audit, add an owner,
# remove an owner, then SELL the dataset. About 1 minute.
python -m demo.run_demo

# STEP 5 – THE BEST ONE TO SHOW. Attacks the older research method and breaks
# it, then runs the identical attack on ours, where it fails. About 1 minute.
python -m attacks.collusion

# STEP 6 – Second attack: somebody faking co-ownership of your data. Blocked.
python -m attacks.rogue_key

# STEP 7 – Third attack: the cloud secretly corrupting data. Always caught.
python -m attacks.tampering

# STEP 8 – Run all 41 tests to prove the code is correct. About 1 minute.
python -m pytest tests/ -v
```

The only real differences

ITEM	MAC	WINDOWS
Python command	<code>python3</code>	<code>python</code>
Installer command	<code>pip3</code>	<code>pip</code>
Slashes in the path	<code>/</code> forward	<code>\</code> backslash
Where to type	Terminal app	Terminal panel inside VS Code

Everything after those four differences is identical on both systems.

What each command produces

COMMAND	TIME	WHAT APPEARS ON SCREEN
<code>datasets.generate</code>	2 sec	Five datasets being written, 125 MB in total
<code>demo.run_demo</code>	1 min	Ten numbered stages, ending in "All 7 protocol checks behaved exactly as expected"
<code>attacks.collusion</code>	1 min	Older method marked BROKEN, this project marked RESISTED
<code>attacks.rogue_key</code>	1 min	Plain method BROKEN, this project RESISTED
<code>attacks.tampering</code>	2 min	A table comparing measured detection rates against theory
<code>pytest tests/ -v</code>	1 min	41 test names, each marked PASSED

Two failures are supposed to happen. In the main demo, stage 6 shows FAIL when the cloud corrupts a block, and stage 9 shows FAIL when the seller tries their old credentials after selling. Both are correct results. The demo prints "as expected" beside each one.

If something goes wrong

```
# PROBLEM: "python is not recognized" (Windows)
# CAUSE: Python is not installed, or was installed without adding it to PATH.
# FIX: install from python.org and TICK THE BOX "Add Python to PATH".
# Or try this alternative launcher:
py -m demo.run_demo

# PROBLEM: "command not found: python" (Mac)
# FIX: on a Mac the command is python3, not python.

# PROBLEM: "No module named crypto" or "No module named datasets"
# CAUSE: you are in the wrong folder. You must be INSIDE the "code" folder.
# Check what folder you are in:
ls      # on Mac
dir     # on Windows
# You should see: crypto, entities, attacks, demo, datasets, tests

# PROBLEM: "no manifest for dataset ..." when running the demo
# CAUSE: STEP 3 was skipped. The demo reads a real dataset from disk.
# FIX: run the datasets.generate command first.

# PROBLEM: "externally-managed-environment" during install (Mac)
# FIX: add this flag to the install command:
pip3 install -r requirements.txt --break-system-packages
```

Most common mistake. Skipping STEP 3. The demo needs the sample datasets to exist before it can run. If the demo stops immediately with a message about a missing manifest, run STEP 3 and then try again.

If you are short on time

Steps 5, 6 and 7 need no setup at all, because they build their own data internally. So if a laptop is being difficult, this single command works straight after STEP 2, and it is the most convincing thing to show:

```
python3 -m attacks.collusion # Mac
python -m attacks.collusion  # Windows
```

It breaks the older published method live, then blocks the identical attack on this project. That single output makes the case faster than any explanation.